

Python MCQ Prep — Theory + Examples (Datagrokr, July 9)

1. Data Types & Mutability

Mutable vs Immutable Mutable objects (list, dict, set) can be changed in place after creation. Immutable objects (int, float, str, tuple, frozenset, bool) can't — any "change" creates a new object.

```
python

a = [1, 2, 3]
b = a
b.append(4)
print(a)  # [1, 2, 3, 4] -- a changed too, same object

x = "hello"
y = x
y += " world"
print(x)  # "hello" -- unaffected, y is a new string object
```

id() behavior `id()` returns an object's memory address. For immutables, `id()` changes when you "modify" the value; for mutables, `id()` stays the same after in-place changes.

```
python

x = 5
print(id(x))
x += 1
print(id(x))  # different id — new int object created

lst = [1, 2]
print(id(lst))
lst.append(3)
print(id(lst))  # same id — modified in place
```

Tuple containing mutable elements A tuple itself can't be reassigned, but if it holds a mutable object like a list, that inner object can still be changed.

```
python

t = (1, [2, 3])
t[1].append(4)
print(t)  # (1, [2, 3, 4]) -- valid, no error
# t[0] = 99 -> TypeError: tuple doesn't support item assignment
```

2. Core Operators & Behavior

Floor division, modulo, power (`//`) gives the integer quotient (rounded down), (`%`) gives the remainder, (`**`) raises to a power.

```
python

print(17 // 5)    # 3
print(17 % 5)     # 2
print(2 ** 10)    # 1024
print(-7 // 2)    # -4 (rounds toward negative infinity, not zero)
```

is vs == (`==`) compares values for equality; (`is`) compares identity (whether both names point to the same object in memory).

```
python

a = [1, 2, 3]
b = [1, 2, 3]
print(a == b)    # True -- same values
print(a is b)    # False -- different objects

c = a
print(a is c)    # True -- same object
```

Short-circuit evaluation (`and`) returns the first falsy operand or the last one; (`or`) returns the first truthy operand or the last one — they don't just return True/False.

```
python

print(0 and 5)    # 0 (first falsy value)
print(3 and 5)    # 5 (both truthy, returns last)
print(0 or 5)     # 5 (first truthy value)
print("" or "default") # "default"
```

Falsy values (`None`), (`False`), (`0`), (`0.0`), (`""`), (`[]`), (`{}`), (`()`) are all falsy in a boolean context, but they are not equal to each other except by explicit numeric comparison.

```
python

print(False == 0)    # True
print(None == False) # False
print(bool([]))      # False
print(bool("0"))     # True -- non-empty string is truthy!
```

3. Strings

Immutability Strings can't be changed in place — every operation that looks like modification (concatenation, replace, upper) returns a brand-new string object.

```
python

s = "abc"
s2 = s.upper()
print(s)    # "abc" -- original unchanged
print(s2)   # "ABC" -- new object
```

Slicing (`s[start:stop:step]`) extracts a substring; negative indices count from the end, and negative step reverses direction.

```
python

s = "HelloWorld"
print(s[0:5])    # "Hello"
print(s[-5:])    # "World"
print(s[::-1])   # "dlroWolleH" -- reversed
print(s[::2])    # "HloWrD" -- every 2nd character
```

split() / join() (`split()`) breaks a string into a list on a delimiter (default whitespace); `join()` does the reverse, gluing a list into one string.

```
python

s = "the quick brown fox"
words = s.split()           # ['the', 'quick', 'brown', 'fox']
print("-".join(words))      # "the-quick-brown-fox"
print("a,b,,c".split(","))  # ['a', 'b', '', 'c'] -- keeps empty strings
```

find() vs index() Both search for a substring's position; `find()` returns -1 if not found, `index()` raises a `ValueError` instead.

```
python

s = "hello world"
print(s.find("xyz"))        # -1
print(s.index("world"))     # 6
# s.index("xyz")            # ValueError: substring not found
```

Case & type-check methods (`.isdigit()`), (`.isalpha()`), (`.isalnum()`) check character composition; useful for input validation MCQs.

```
python

print("123".isdigit())      # True
print("abc123".isalnum())   # True
print("abc 123".isalnum())  # False -- space isn't alphanumeric
```

f-strings and formatting f-strings embed expressions directly in a string with `{}`, and support format specs like `.2f` for decimal precision.

```
python
```

```
x = 3.14159
print(f"{x:.2f}")          # "3.14"
name, score = "Ravi", 92
print(f"{name} scored {score}") # "Ravi scored 92"
```

4. Lists

append() vs extend() `.append()` adds its argument as a single element (even if it's a list); `.extend()` unpacks an iterable and adds each item individually.

```
python
```

```
a = [1, 2]
a.append([3, 4])
print(a) # [1, 2, [3, 4]] -- nested list added as one item

b = [1, 2]
b.extend([3, 4])
print(b) # [1, 2, 3, 4] -- items added individually
```

insert() / pop() / remove() `.insert(i, x)` adds at index *i*; `.pop(i)` removes and returns the item at index *i* (default last); `.remove(x)` deletes the first matching value and errors if absent.

```
python
```

```
lst = [10, 20, 30]
lst.insert(1, 15)      # [10, 15, 20, 30]
removed = lst.pop()    # removed = 30, lst = [10, 15, 20]
lst.remove(15)         # lst = [10, 20]
# lst.remove(99)       # ValueError: not in list
```

sort() vs sorted() `.sort()` sorts a list in place and returns `None`; `sorted()` returns a new sorted list, leaving the original untouched.

```
python
```

```
a = [3, 1, 2]
result = a.sort()
print(a, result)      # [1, 2, 3] None

b = [3, 1, 2]
c = sorted(b)
print(b, c)           # [3, 1, 2] [1, 2, 3]
```

Shallow copy via slicing `old[:]` creates a new list object with the same top-level elements — changes to the copy's structure don't affect the original, but nested mutable elements are still shared.

```
python

a = [1, 2, 3]
b = a[:]
b.append(4)
print(a, b)  # [1, 2, 3] [1, 2, 3, 4] -- independent

# but nested objects are shared:
x = [[1, 2], [3, 4]]
y = x[:]
y[0].append(99)
print(x)  # [[1, 2, 99], [3, 4]] -- inner list changed in both
```

count() / index() `.count(x)` returns how many times `x` appears; `.index(x)` returns the position of the first occurrence.

```
python

scores = [50, 25, 50, 50, 10]
print(scores.count(50))  # 3
print(scores.index(50))  # 0
```

Nested list multiplication trap `[[0]*3]*3` creates 3 references to the *same* inner list, so modifying one row changes all rows — a classic MCQ trap.

```
python

grid = [[0]*3]*3
grid[0][0] = 1
print(grid)  # [[1, 0, 0], [1, 0, 0], [1, 0, 0]] -- all rows changed!

# correct way:
grid2 = [[0]*3 for _ in range(3)]
grid2[0][0] = 1
print(grid2)  # [[1, 0, 0], [0, 0, 0], [0, 0, 0]]
```

5. Dictionaries

dict[key] vs .get() Direct indexing raises `KeyError` if the key is missing; `.get(key, default)` returns a fallback value instead, avoiding a crash.

```
python
```

```
d = {"a": 1, "b": 2}
print(d.get("c", 0))    # 0 -- safe
# print(d["c"])          # KeyError
```

keys() / values() / items() These give view objects over a dict's keys, values, or (key, value) pairs — commonly used to iterate.

```
python
```

```
d = {"x": 10, "y": 20}
for k, v in d.items():
    print(k, v)
# x 10
# y 20
```

setdefault() / update() `.setdefault(key, default)` returns the value if present, otherwise inserts and returns the default; `.update()` merges another dict/pairs into the current one.

```
python
```

```
d = {"a": 1}
d.setdefault("b", 5)
print(d)    # {'a': 1, 'b': 5}

d.update({"a": 100, "c": 3})
print(d)    # {'a': 100, 'b': 5, 'c': 3}
```

Dict comprehension Builds a new dict in one line using `{key_expr: value_expr for item in iterable}`.

```
python
```

```
squares = {x: x**2 for x in range(5)}
print(squares)    # {0: 0, 1: 1, 2: 4, 3: 9, 4: 16}
```

Insertion order (Python 3.7+) Since Python 3.7, dictionaries maintain the order in which keys were inserted — this is now guaranteed language behavior, not an implementation detail.

```
python
```

```
d = {}
d["z"] = 1
d["a"] = 2
print(list(d.keys()))    # ['z', 'a'] -- insertion order preserved
```

6. Functions & Scope

Mutable default argument trap A mutable default argument (like a list) is created *once* when the function is defined, not each call — so it persists and accumulates across calls, a very common MCQ gotcha.

```
python

def add_item(x, lst=[]):
    lst.append(x)
    return lst

print(add_item(1))  # [1]
print(add_item(2))  # [1, 2] -- unexpected! same list reused

# fix:
def add_item_fixed(x, lst=None):
    if lst is None:
        lst = []
    lst.append(x)
    return lst
```

***args and **kwargs** `(*args)` collects extra positional arguments into a tuple; `(**kwargs)` collects extra keyword arguments into a dict.

```
python

def demo(*args, **kwargs):
    print(args)      # tuple
    print(kwargs)    # dict

demo(1, 2, name="Ravi", age=21)
# (1, 2)
# {'name': 'Ravi', 'age': 21}
```

Local vs global scope Variables assigned inside a function are local by default; use `global` to modify a module-level variable from within a function.

```
python

count = 0
def increment():
    global count
    count += 1

increment()
print(count)  # 1
```

Lambda functions A lambda is an anonymous, single-expression function, often used inline with `map`, `filter`, or `sorted(key=...)`.

```
python

square = lambda x: x**2
print(square(5))  # 25

pairs = [(1, 'b'), (2, 'a')]
print(sorted(pairs, key=lambda p: p[1]))  # [(2, 'a'), (1, 'b')]
```

7. Control Flow

for-else / while-else The `else` block runs only if the loop completes normally (no `break`) — useful for "search and confirm not found" logic.

```
python

for i in range(5):
    if i == 10:
        break
else:
    print("Loop completed without break")  # this prints
```

try-except-else-finally `except` runs only on error; `else` runs only if no exception occurred; `finally` always runs, regardless of what happened.

```
python

try:
    x = 10 / 2
except ZeroDivisionError:
    print("Error!")
else:
    print("No error, x =", x)  # runs since no exception
finally:
    print("This always runs")
```

range() stop exclusion `range(start, stop, step)` generates numbers up to but not including `stop`.

```
python

print(list(range(2, 10, 2)))  # [2, 4, 6, 8] -- 10 excluded
```

8. Iterables

zip(), enumerate() (`zip()`) pairs up elements from multiple iterables positionally; (`enumerate()`) pairs each element with its index.

```
python

names = ["A", "B", "C"]
scores = [90, 85, 70]
for name, score in zip(names, scores):
    print(name, score)

for i, name in enumerate(names):
    print(i, name)

# 0 A
# 1 B
# 2 C
```

map(), filter() (`map(func, iterable)`) applies a function to every item; (`filter(func, iterable)`) keeps only items where the function returns True.

```
python

nums = [1, 2, 3, 4, 5]
squared = list(map(lambda x: x**2, nums))          # [1, 4, 9, 16, 25]
evens = list(filter(lambda x: x % 2 == 0, nums))  # [2, 4]
```

sum, min, max, sorted with key These aggregate or reorder iterables; (`sorted(..., key=..., reverse=...)`) allows custom sort logic.

```
python

people = [("Ravi", 22), ("Asha", 19), ("Kiran", 25)]
print(sorted(people, key=lambda p: p[1]))          # sort by age asc
print(sorted(people, key=lambda p: p[1], reverse=True)) # desc
print(max(people, key=lambda p: p[1]))            # ('Kiran', 25)
```

9. OOP Basics

class, `__init__`, self A class is a blueprint for objects; (`__init__`) is the constructor that runs when an object is created, and (`self`) refers to the specific instance.

python

```
class Student:
    def __init__(self, name, marks):
        self.name = name
        self.marks = marks

s1 = Student("Ravi", 90)
print(s1.name, s1.marks)  # Ravi 90
```

Instance vs class variables Instance variables are unique per object (defined via `self.x`); class variables are shared across all instances of the class.

python

```
class Counter:
    total = 0  # class variable, shared

    def __init__(self):
        Counter.total += 1
        self.id = Counter.total  # instance variable, unique

c1 = Counter()
c2 = Counter()
print(c1.id, c2.id, Counter.total)  # 1 2 2
```

Inheritance and super() A subclass inherits attributes/methods from a parent class; `super()` lets the child call the parent's methods, commonly used inside `__init__`.

python

```
class Animal:
    def __init__(self, name):
        self.name = name
    def speak(self):
        return f"{self.name} makes a sound"

class Dog(Animal):
    def speak(self):
        return f"{self.name} barks"

d = Dog("Rex")
print(d.speak())  # Rex barks
```

10. Worked Practice Problems (matching reported exam patterns)

A. Frequency counting

python

```
def count_occurrences(lst, target):  
    return lst.count(target)  
  
print(count_occurrences([50, 25, 50, 50, 10], 50)) # 3
```

B. Mean + classify (two-pass pattern — their most repeated question type)

python

```
def classify_by_mean(records):  
    # records = [(name, marks), ...]  
    mean = sum(m for _, m in records) / len(records)  
    result = []  
    for name, marks in records:  
        flag = 1.00 if marks < mean else 0.00  
        result.append((name, flag))  
    return result  
  
data = [("A", 80), ("B", 40), ("C", 60)]  
print(classify_by_mean(data))  
# [('A', 0.0), ('B', 1.0), ('C', 0.0)] -- mean=60, B is below
```

C. Longest substring of vowels (sliding window)

python

```
def longest_vowel_substring(s):  
    vowels = set("aeiouAEIOU")  
    best_start, best_len, cur_start = 0, 0, 0  
    for i, ch in enumerate(s):  
        if ch not in vowels:  
            cur_start = i + 1  
        elif i - cur_start + 1 > best_len:  
            best_len = i - cur_start + 1  
            best_start = cur_start  
    return s[best_start:best_start + best_len], best_len  
  
print(longest_vowel_substring("xaeioubcdaeioxyz")) # ('aeiou', 5)
```

D. Remove duplicates, preserve order

python

```
def dedupe(lst):
    seen = set()
    result = []
    for x in lst:
        if x not in seen:
            seen.add(x)
            result.append(x)
    return result

print(dedupe([1, 2, 2, 3, 1, 4])) # [1, 2, 3, 4]
```

E. Merge dicts, summing common keys

python

```
def merge_sum(d1, d2):
    result = dict(d1)
    for k, v in d2.items():
        result[k] = result.get(k, 0) + v
    return result

print(merge_sum({"a": 1, "b": 2}, {"b": 3, "c": 4}))
# {'a': 1, 'b': 5, 'c': 4}
```

F. Mean, median, mode without libraries

python

```
def mean(lst):
    return sum(lst) / len(lst)

def median(lst):
    s = sorted(lst)
    n = len(s)
    mid = n // 2
    if n % 2 == 0:
        return (s[mid-1] + s[mid]) / 2
    return s[mid]

def mode(lst):
    counts = {}
    for x in lst:
        counts[x] = counts.get(x, 0) + 1
    return max(counts, key=counts.get)

data = [4, 2, 7, 2, 9, 2, 4]
print(mean(data), median(data), mode(data)) # 4.28... 4 2
```

